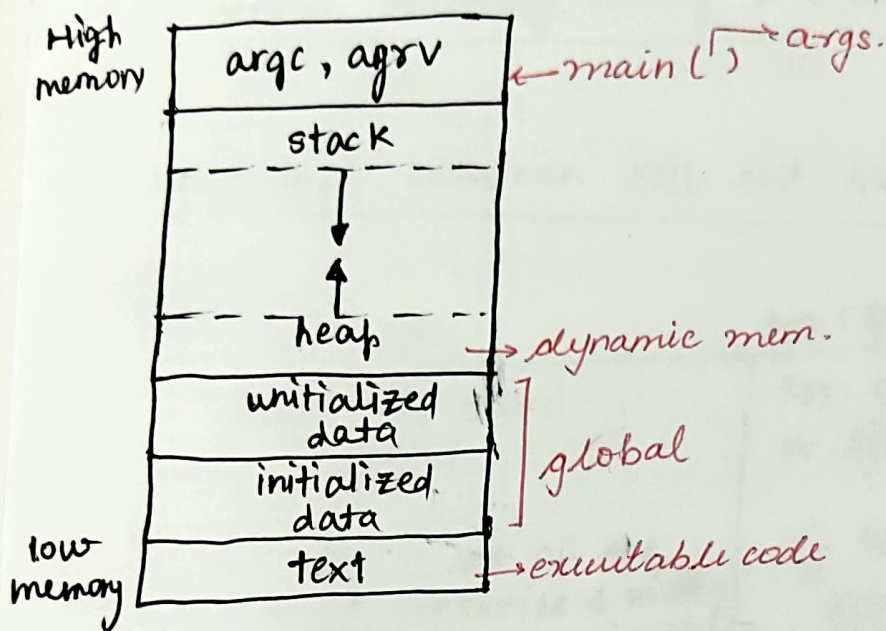
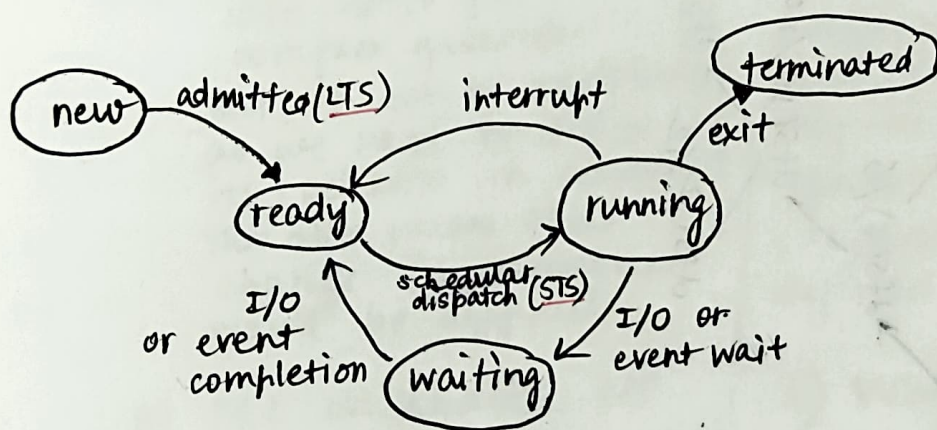


Memory Layout

a prog in execution



PROCESS STATE DIAGRAM



LTS - Long term scheduler

STS - Short term scheduler

PCB (Process Control Block)

Process state
Process Number
Program Counter (PC)
Registers
Memory Limits
List of open Files

Degree of Multiprogramming

→ No. of processes in the ready queue.

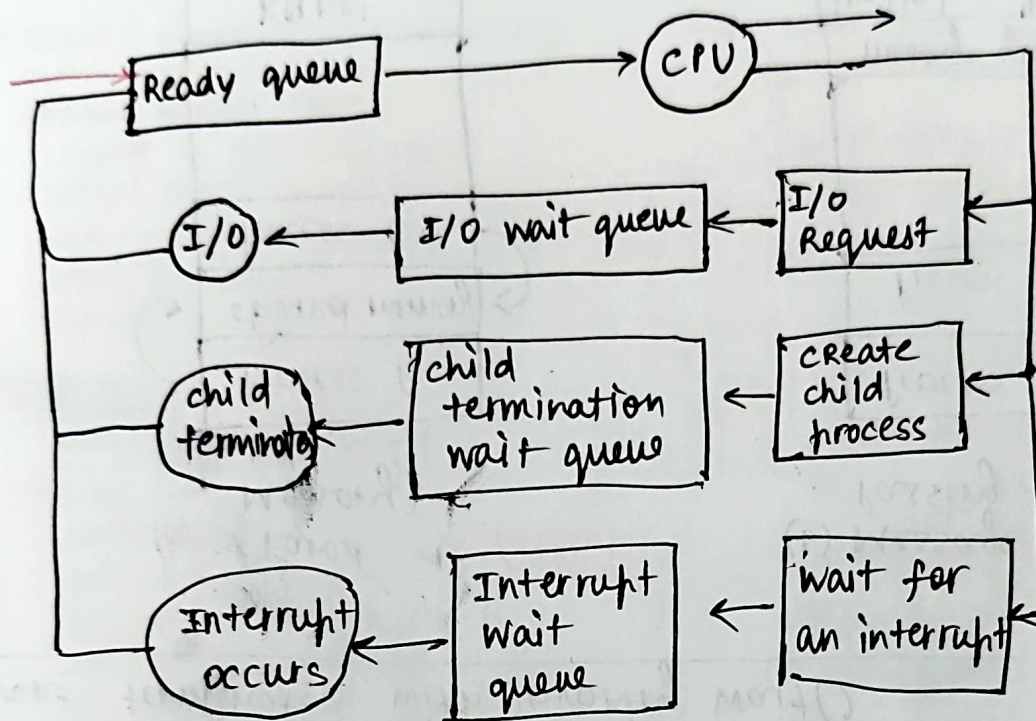
REPRESENTATION OF PROCESS SCHEDULING

all active processes



task_struct.
(doubly ll)

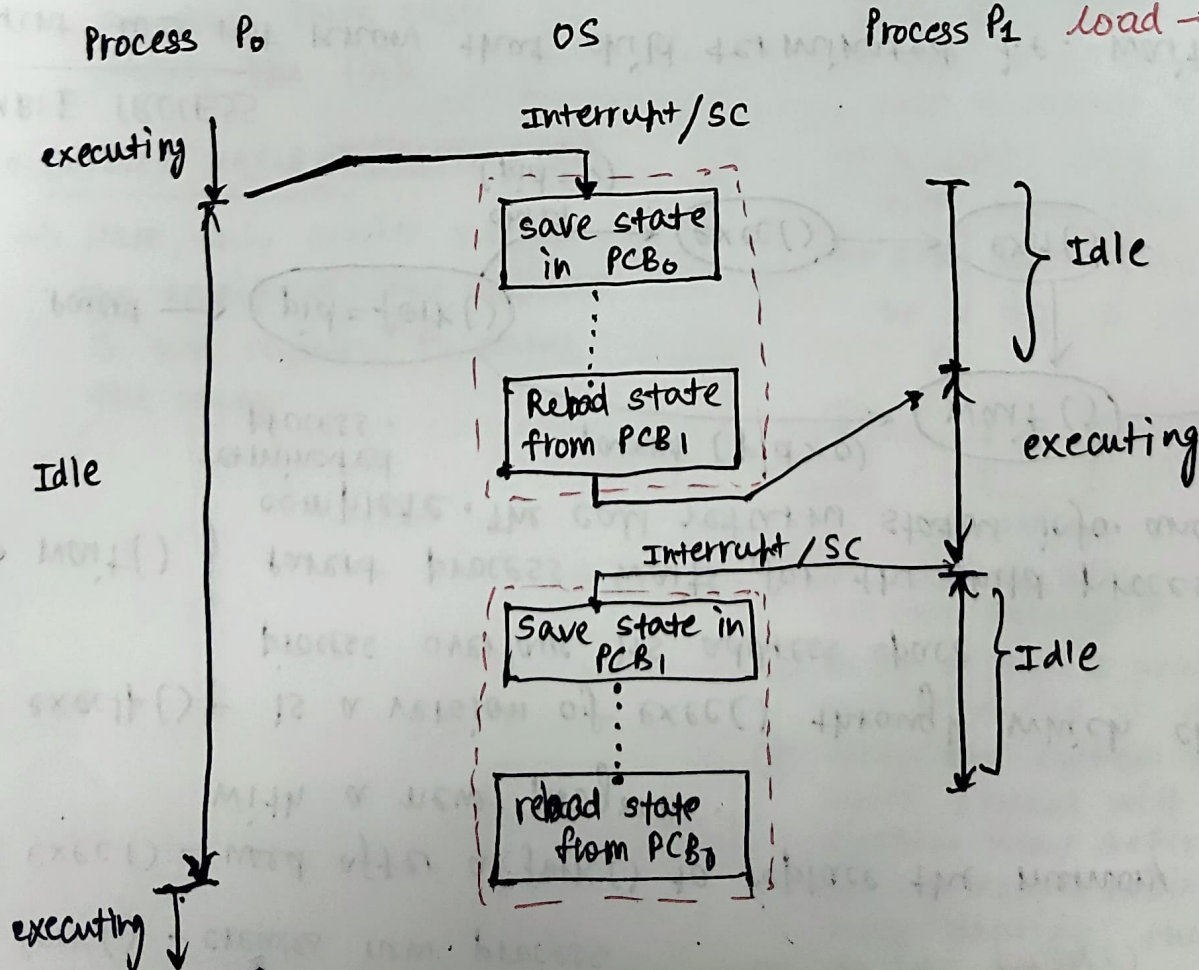
ea queue
[ready } head,
[waiting } tail



CONTEXT SWITCH DIAGRAM B/W TWO PROCESSES

(pure overhead)

save → old p
load → new p.



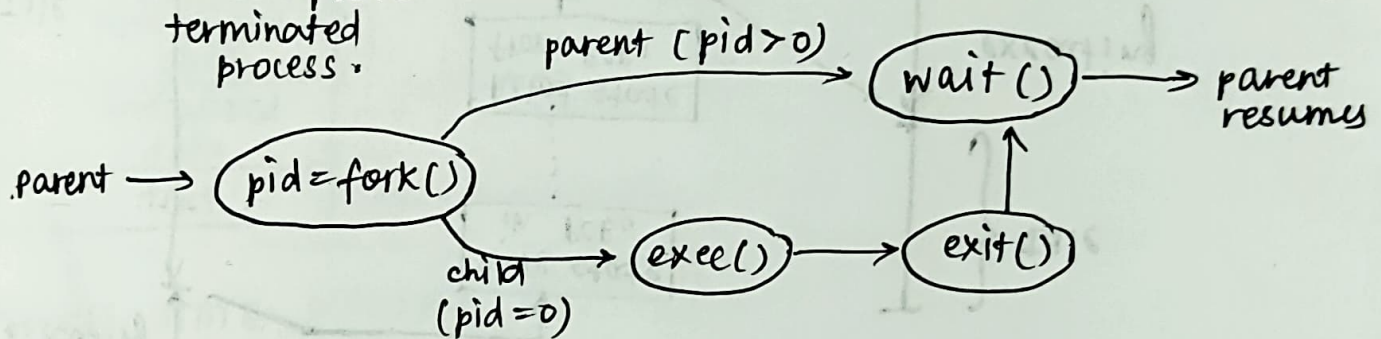
→ `fork()` - creates new process

return()

→ `exec()` - used after a `fork()` to replace the memory space with a new program.

→ `execlp()` - is a version of `exec()` through which child process overlays its address space

→ `wait()` - parent process waits for the child process to complete. The call returns status info. and pid of



ZOMBIE PROCESS

→ Parent does not know that child terminated i.e. `wait()` was not invoked.

ORPHAN PROCESS

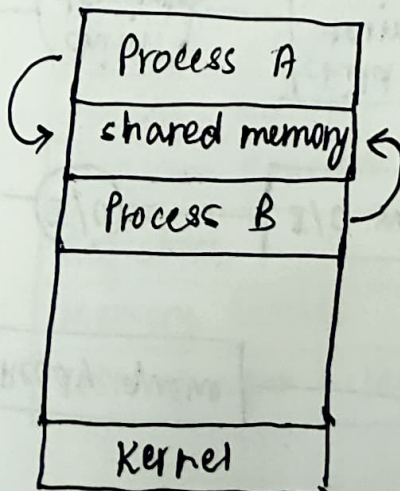
→ Parent terminated with invoking `wait()`.

cascading termination

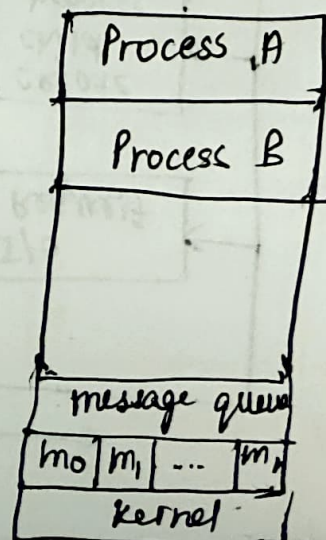
Android
foreground background
↓
services

IPC

(a) Shared Memory *(faster)*



(b) Message Passing.



MESSAGE PASSING

(no shared vars)

NAMING

DIRECT COMMUNI. (SYMMETRY)

- Each process that wants to communicate must explicitly name the recipient or sender.
- send(P, message)
- receive(Q, message)

PROPERTIES OF COMMUN. LINK

- Link is established automatically b/w every pair of processes. The processes need to know each other's identity to communicate.
- A link is associated with exactly two processes.
- B/w each pair of processes, there exists exactly one link.

ANOTHER VARIANT (ASYMMETRY)

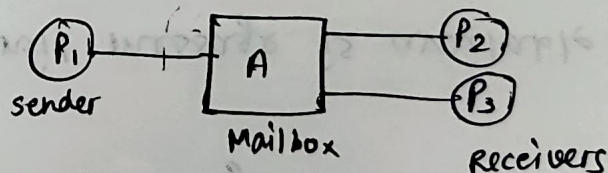
- Here, only sender names the recipient, the recipient is not required to name the sender.

INDIRECT COMMUNI.

- Messages are sent to and received from Mailboxes.
- A mailbox can be viewed abstractly as an object into which messages can be placed by processes and from which messages can be removed.
- Each mailbox has a unique id.
- * → Two processes can communicate if they have a shared mailbox.

PROP. OF COMMUN. LINK

- Link is established if they have a shared mailbox.
- A Link may be associated with more than two processes.
- B/w each pair, there may be a no. of diff. links.



Which process will receive message?

- ① Allow at most 1 process to execute receive op. at ~~the~~ ^a time.
- ② allow the system to decide which process will receive. The system may define an algorithm for e.g. Round Robin. The system may identify the receiver to the sender.

→ synchronization

→ Message passing may be either blocking or non-blocking, also known as synchronous & asynchronous.

→ BLOCKING SEND (SYNCH.) :- sending process is blocked until the message is received by the receiving process or by the mailbox.

→ NON BLOCKING
SEND (ASYNCH) :- Sending process sends the message & resumes operation.

→ BLOCKING RECEIVE :- Receiver blocks until message is available.

→ NON BLOC : Receiver retrieves either a valid message or a null.

→ BUFFERING → Messages exchanged by communicating process (Dir. & Indir) reside in a temporary queue. Queue is implemented in following three ways :-

1. Zero capacity: Basically $q.size() = 0$ i.e. After sending a message, sender has to wait until the message is received to send the next one.

2. Bounded :- finite length of n message.

3. Unbounded - sender never waits.

→ Example of IPC - Windows.

↳ message passing centric via ALPC

Advanced Local Procedure Call

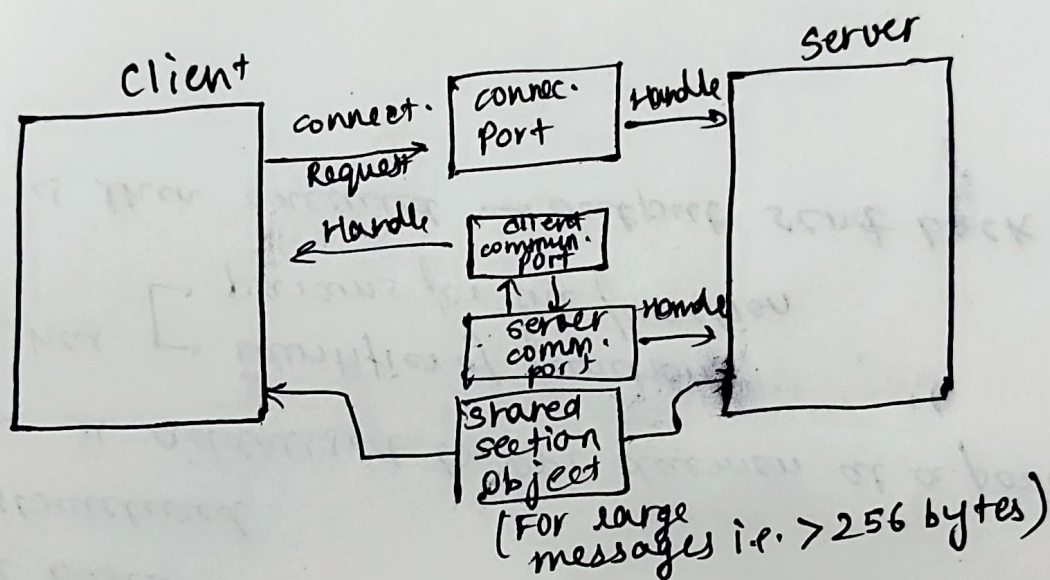
ALPC

→ Works b/w processes on the same system.

→ Uses ports to establish & maintain communication channels.

HOW COMMUNICATION WORKS IN ALPC :-

- ① client (Application Program) opens a handle to subsystem's connection port object.
 - ② client sends connection req.
 - ③ server creates 2 private commu. ports, one of which it gives to client. One for client-server message, other for server-client.
 - ④ client & server use the corresp. handles to send mess.
- For small messa., message queue is used
→ For large " , they must be passed through Section Object.



PIPES

ordinary pipes → parent-child

Named pipes → x "

ORDINARY PIPES

- Producer-Consumer Style
- unidirectional
- windows → anonymous pipes
- ~~unix~~ UNIX → pipes ≡ files. (read, write)

SOCKETS

endpoint for communication.

IP + Port

↳ diff network services on a host

- Java types:
- Connection Oriented (TCP)
 - Connectionless (UDP)
 - Multicast Sockets

Remote Procedural Calls (RPC)

- diff. systems over network. → request service
- message based
- well structured
- message is addressed to RPC daemon at a port ^{always listening}
- message has
 - ↳ identifier of function
 - ↳ params for the function
- function is then executed → output sent back ☑